

A PROJECT REPORT ON

**Kloudraksha: A Cloud Auditing Tool for
Enhanced Security and Compliance**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

AMEYA SURANA
MUSTAFA TRUNKWALA
RIYA WANI

EXAM NO. B400050605
EXAM NO. B400050618
EXAM NO. B400050638



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled

Kloudraksha: A Cloud Auditing Tool for Improved Security and Compliance

Submitted by

AMEYA SURANA
MUSTAFA TRUNKWALA
RIYA WANI

EXAM NO. B400050605
EXAM NO. B400050618
EXAM NO. B400050638

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Bhumesh Masram** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. Bhumesh Masram
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 16/10/2024

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance**’.*

*We would like to thank our project guide **Prof. Bhumesh Masram** as well as subject experts **Prof. R. S. Pasawan** and **Prof. Y. A. Handge** for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

*We are also grateful to **Dr G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

*In the end our special thanks to our mentors **Mr. Mandar Waghmare**, Newton’s Apple for their continued support and sponsorship of the problem statement. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Ameya Surana
Mustafa Trunkwala
Riya Wani
(B.E. Computer Engg.)

ABSTRACT

As cloud adoption grows, the security and compliance landscape becomes increasingly complex, especially for organisations managing multi-cloud environments. Misconfigurations in cloud infrastructure are a leading cause of data breaches, and existing cloud auditing tools often do not address this issue comprehensively. Most current solutions lack multi-cloud compatibility, rely on manual or periodic audits, and present complex technical reports that are not easily actionable for business stakeholders. This survey paper explores the limitations of existing cloud auditing tools and proposes Kloudraksha, an innovative cloud auditing tool designed to provide real-time, automated, and cross-platform cloud audits. Kloudraksha offers continuous monitoring of cloud configurations, detects security vulnerabilities, and ensures compliance with various regulatory standards such as GDPR, HIPAA, and PCI-DSS. Using AI / ML algorithms, Kloudraksha not only identifies misconfigurations, but also suggests or automates remediation actions. The intuitive dashboard of the tool provides nontechnical users with simplified reports and actionable insights, empowering businesses to maintain security and compliance efficiently. The paper reviews existing research and tools in the cloud auditing domain, identifies key gaps, and presents the architecture and implementation of Kloudraksha. By integrating with major cloud platforms like AWS, Azure, and Google Cloud, Kloudraksha enhances the security posture of organizations while simplifying the complexity of cloud compliance. Through the proposed solution, our aim is to revolutionize how businesses approach cloud configuration audits and respond to the evolving challenges of cloud security.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodology of Problem solving	3
2	Literature Survey	4
2.1	Literature survey	5
3	Software Requirements Specification	7
3.1	Assumptions and Dependencies	8
3.2	Functional Requirements	8
3.2.1	User Authentication and Role Management	8
3.2.2	Multi-Cloud Integration	8
3.2.3	Real-Time Monitoring and Auditing	8
3.2.4	Compliance Validation	9
3.2.5	Automated Remediation	9
3.2.6	Report Generation and Visualization	9
3.2.7	User Dashboard	9
3.3	External Interface Requirements	10
3.3.1	Software Interfaces	10
3.4	Nonfunctional Requirements	11
3.4.1	Performance Requirements	11
3.4.2	Safety Requirements	11
3.5	Scalability	11
3.5.1	Reliability and Availability	11
3.5.2	Security	11
3.5.3	Compliance	12
3.6	Analysis Models	12
3.6.1	SDLC Model	12
4	System Design	13
4.1	Model Architecture	14
4.2	Key Components	15
4.2.1	Multi-cloud Integration Layer	15
4.2.2	Real-time Monitoring Engine	15

4.2.3	Compliance Engine	15
4.2.4	User Dashboard	15
4.2.5	Authentication and User Management	15
4.3	Data Flow	15
4.3.1	Data Collection	15
4.3.2	Monitoring and Analysis	16
4.3.3	Compliance Checks	16
4.3.4	Remediation Actions	16
4.3.5	User Notification	16
4.3.6	Report Generation	16
4.4	Deployment and Scalability	16
4.5	Security Considerations	16
5	Project Plan	17
5.1	UML Diagrams	18
5.1.1	Class Diagram	18
5.1.2	Use Case Diagram	18
5.1.3	Sequence Diagram	18
5.2	Project Estimate	20
5.2.1	Reconciled Estimates	20
5.2.2	Project Resources	21
5.2.3	Project Timeline	22
5.3	Risk Management	23
5.3.1	Risk Identification	23
5.3.2	Risk Analysis	23
5.3.3	Overview of Risk Mitigation, Monitoring, Management	24
5.3.4	Project Task Set	25
5.4	Team Organization	25
5.4.1	Team structure	25
5.4.2	Management reporting and communication	26
6	Project Implementation	27
6.1	Overview of Project Modules	28
6.2	Tools and Technologies Used	29
6.3	Algorithm Details	29
7	Software Testing	31
7.1	Type of Testing	32
7.2	Test cases & Test Results	33
8	Results	36
8.1	Outcomes	37
9	Conclusions	45
9.1	Conclusions	46

10	Appendix	47
10.1	Appendix A	48
	10.1.1 Survey Paper Details	48
	10.1.2 Implementation Paper Details	48
10.2	Appendix B	48
	10.2.1 Plagarism Report	48
11	References	49

List of Figures

3.1	SDLC Model	12
4.1	System Architecture	14
5.1	Class Diagram	18
5.2	Use Case Diagram	19
5.3	Sequence Diagram	20
5.4	Project Timeline	22
8.1	Hero section of website	39
8.2	Login Screen	39
8.3	User Dashboard	40
8.4	User Dashboard - My Scan	40
8.5	User Dashboard - New Scan	41
8.6	Admin Dashboard - Kanban Layout	41
8.7	Audit Dashboard	42
8.8	Super Admin Dashboard	42
8.9	Super Admin Dashboard - User Management	43
8.10	Super Admin Dashboard - Audit Overview	43
8.11	Super Admin Dashboard - Platform Logs	44
8.12	Subscription Model	44
10.1	Plagiarism Report	48

LIST OF ABBREVIATIONS

Abbreviation	Illustration
API	Application Programming Interface
AWS	Amazon Web Services
GCP	Google Cloud Platform
ML	Machine Learning
AI	Artificial Intelligence
SLA	Service Level Agreement
JSON	JavaScript Object Notation
HIPAA	Health Insurance Portability and Accountability Act
PCI-DSS	Payment Card Industry Data Security Standard
GDPR	General Data Protection Regulation
CLI	Command Line Interface
SDK	Software Development Kit
VPC	Virtual Private Cloud

CHAPTER 1

INTRODUCTION

1.1 Overview

Cloud computing has transformed how businesses operate by providing scalable, flexible, and cost-efficient resources. However, with these advantages come new security and compliance challenges. Organisations must ensure their cloud infrastructure complies with industry standards such as CIS, GDPR, HIPAA, and PCI-DSS while minimising security vulnerabilities due to misconfigurations. As more businesses adopt multi-cloud strategies, the complexity of managing and securing cloud environments increases significantly.

1.2 Motivation

Existing cloud auditing tools often lack automation, are confined to single-cloud environments, and provide complex reports that are not actionable for all stakeholders. Many tools focus on periodic audits rather than continuous real-time monitoring, leading to delayed detection of issues. These tools also fail to offer comprehensive compliance support for different industry regulations across multi-cloud environments. Kloudraksha aims to redefine cloud configuration audits by offering a real-time, automated, and multi-cloud auditing tool. The goal is to simplify cloud audits and empower organisations to ensure cloud security and compliance more effectively.

1.3 Problem Definition and Objectives

As businesses increasingly adopt cloud environments, managing security and ensuring compliance across multi-cloud infrastructures has become a significant challenge. Misconfigurations in cloud services, inadequate compliance with regulatory standards, and the complexity of manually auditing cloud infrastructures expose organizations to severe risks, including data breaches, security vulnerabilities, and non-compliance penalties. Existing cloud auditing tools often lack multi-cloud support, rely on periodic manual audits, and produce overly technical reports that are difficult for non-technical stakeholders to interpret and act upon. This creates a need for an automated, real-time, multi-cloud auditing solution that can simplify compliance management, ensure cloud security, and deliver actionable insights for businesses.

The primary objective of this research and the development of Kloudraksha is to create a comprehensive cloud auditing solution that addresses the challenges organizations face in managing cloud security and compliance. Specifically, Kloudraksha aims to provide a multi-cloud auditing tool that seamlessly integrates with AWS, Azure, Google Cloud, and other platforms, enabling real-time, automated audits that continuously monitor for security vulnerabilities, misconfigurations, and compliance violations. The tool will

simplify compliance management by supporting multiple regulatory frameworks and offering customizable compliance checks tailored to specific industry needs. Additionally, Kloudraksha will prioritize delivering actionable, user-friendly reports that are accessible to both technical and non-technical users, while leveraging AI-driven automation to streamline remediation processes and enhance overall cloud security. Designed with scalability in mind, Kloudraksha will be capable of supporting large and dynamic cloud environments, empowering enterprises to manage their growing cloud infrastructures with confidence and efficiency.

1.4 Project Scope & Limitations

The scope of the Kloudraksha project focuses on developing a multi-cloud auditing solution that integrates with major cloud platforms like AWS, Azure, and Google Cloud, providing real-time monitoring, continuous audits, and comprehensive compliance management across cloud environments. The tool aims to simplify security and compliance by offering AI-driven automation, customizable compliance checks, and actionable insights through user-friendly reports accessible to both technical and non-technical users. Designed to scale with growing infrastructures, Kloudraksha will also integrate with existing DevOps pipelines to ensure seamless auditing throughout the development lifecycle. However, its limitations include a strong dependency on cloud provider APIs, meaning any changes or restrictions in these APIs could impact the tool's functionality or require updates to maintain compatibility.

1.5 Methodology of Problem solving

The methodology for solving the cloud security and compliance challenges in the Kloudraksha project involves several phases. First, a detailed analysis of the current cloud auditing challenges is conducted, focusing on multi-cloud environments, security misconfigurations, and gaps in compliance management. Following this, a literature review and market analysis are performed to identify the limitations of existing tools and methodologies, such as lack of scalability and real-time monitoring. Based on these insights, Kloudraksha's architecture is designed to support multi-cloud integration, real-time auditing, AI-driven automation, and customizable compliance frameworks. The solution is then developed with a focus on scalability and efficiency, incorporating AI/ML algorithms for automating security checks and remediation processes. Testing and evaluation of the system are conducted to ensure that it meets the performance, security, and compliance needs of organizations, allowing for continuous cloud auditing and seamless integration with DevOps pipelines. This phased approach ensures that the tool addresses both the technical and business challenges associated with cloud security auditing.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature survey

Cloud computing has revolutionized the IT landscape by providing scalable, on-demand resources, but it has also introduced significant security challenges, particularly around configuration management and compliance auditing. Misconfigurations and security lapses can lead to data breaches, unauthorized access, and regulatory non-compliance. Auditing cloud environments—whether through periodic or continuous monitoring—is critical for ensuring both security and adherence to industry standards. In this context, tools and frameworks like Kloudraksha aim to provide comprehensive solutions for continuous auditing, ensuring that businesses can manage their cloud infrastructure effectively without sacrificing security. This paper presents a literature survey of continuous cloud auditing methodologies, focusing on key innovations and the evolving challenges in cloud security auditing.

1. Continuous Auditing in Multi-Cloud Environments: Cloud environments, especially multi-cloud infrastructures, introduce unique security challenges due to their complexity and dynamic nature. Torkura et al. (2022) present CSBAuditor, a system designed for continuous auditing and threat detection across multi-cloud setups. Their work emphasizes the importance of secure baseline configurations and automated monitoring to detect misconfigurations early. The system also integrates state transition analysis, ensuring that cloud systems remain secure even when rapid changes occur in infrastructure.

2. Cloud Security Auditing: Methodologies and Challenges: Another major study by Ligatti et al. (2016) explores different cloud security auditing approaches, particularly focusing on traditional auditing techniques and their scalability issues. This paper highlights the shortcomings of point-in-time audits in modern cloud infrastructures, which require real-time oversight due to their dynamic and often unpredictable nature. The authors advocate for a shift toward continuous auditing practices, which utilize automated tools and advanced security protocols to manage security risks effectively.

3. Security and Privacy in Cloud Data Auditing: Focusing on the privacy aspects of cloud auditing, Wang et al. (2023) present a comprehensive review of cryptographic techniques like homomorphic verifiable tags (HVTs) and message authentication codes (MACs) that are used to maintain data integrity during audits. These methods allow auditors to verify the integrity of outsourced data without compromising user privacy. The paper also discusses the limitations of third-party auditors in maintaining data confidentiality, proposing more secure, privacy-preserving protocols.

4. The MEDINA Framework: From Point-in-Time to Continuous Auditing: A major leap in cloud auditing is the transition from periodic to continuous auditing, as highlighted by the MEDINA framework.

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

This framework addresses the limitations of traditional audit systems by implementing continuous monitoring, automated evidence collection, and Blockchain-based tracking to ensure audit trail integrity. The MEDINA framework is particularly suited for organizations aiming to maintain compliance with industry standards like ISO 27001 through continuous certification.

5. AMOE: Automating Organizational Evidence Extraction for Cloud Audits: An innovative approach to automating the audit process is introduced by AMOE, a tool that leverages Natural Language Processing (NLP) to extract and assess organizational evidence from cloud environments. This system improves the accuracy and efficiency of auditing by automating the analysis of large volumes of textual data, such as policy documents and compliance reports. It enhances the ability of auditors to detect and assess compliance with security policies in real-time.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

Kloudraksha operates under several key assumptions and dependencies to ensure its functionality and performance. It assumes that the organization is using a multicloud environment, specifically platforms like AWS, Azure, and Google Cloud, with accessible APIs for seamless integration. Full API access and appropriate permissions are required to perform cloud configuration audits, relying on the API rate limits set by the cloud providers. Real-time or near real-time data flows are necessary for continuous auditing and monitoring, and stable internet connectivity is essential for communication between the tool and cloud platforms. The tool also assumes cloud provider APIs will remain stable; significant changes might require tool updates. Kloudraksha relies on predefined libraries for compliance standards, such as GDPR and HIPAA, and any updates to these standards would necessitate updates to the tool. The infrastructure must support Redis and Vector DB for fast data storage and querying, ensuring efficient real-time performance. Furthermore, machine learning algorithms depend on up-to-date libraries to maintain reliable anomaly detection and predictive auditing capabilities.

3.2 Functional Requirements

3.2.1 User Authentication and Role Management

Kloudraksha must provide a secure authentication system, allowing user login via multi-factor authentication. Role-based access control (RBAC) must be implemented to restrict access to sensitive data and audit results.

Input: User login credentials.

Output: Dashboard access based on user roles (e.g., Admin, Auditor, Viewer).

3.2.2 Multi-Cloud Integration

Kloudraksha must integrate with multiple cloud platforms (AWS, Azure, Google Cloud) through their APIs. It should retrieve real-time configuration data across these environments.

Input: API credentials from cloud providers.

Output: Continuous data feeds from multiple cloud platforms.

3.2.3 Real-Time Monitoring and Auditing

The system must continuously monitor cloud configurations, detect misconfigurations, and identify vulnerabilities in real time using predefined rules and AI algorithms.

Input: Cloud configuration data, machine learning models.

Output: Real-time alerts for misconfigurations and vulnerabilities.

3.2.4 Compliance Validation

Kloudraksha must validate cloud configurations against industry-specific compliance standards (e.g., GDPR, HIPAA, PCI-DSS). It should allow custom compliance checks for organizational needs.

Input: Cloud configuration data, compliance standards.

Output: Reports showing compliance status and violations.

3.2.5 Automated Remediation

The system must suggest automated remediation actions for identified security risks and compliance violations. Where applicable, it should apply predefined remediation automatically.

Input: Misconfiguration alerts, security vulnerabilities.

Output: Suggested actions or automatic fixes applied.

3.2.6 Report Generation and Visualization

Kloudraksha must generate intuitive, easy-to-understand audit reports, providing visualizations and summaries for both technical and non-technical users.

Input: Audit data, compliance status, remediation actions.

Output: Visualized audit reports with actionable insights.

3.2.7 User Dashboard

The user dashboard must provide a real-time overview of security audits, compliance status, vulnerabilities, and remediation actions, tailored to user roles.

Input: Audit results, compliance data.

Output: Customizable dashboard views with key metrics.

3.3 External Interface Requirements

- User Interfaces.
- Application Programming Interfaces (APIs)
- Hardware Interfaces
- Software Interfaces.
- Communication Interfaces.

3.3.1 Software Interfaces

Kloudraksha's software interfaces are designed to ensure seamless integration with various cloud platforms and supporting tools, enabling comprehensive cloud auditing and security management. It connects to cloud service providers like AWS, Azure, and Google Cloud through their respective SDKs and APIs, allowing real-time access to cloud configurations and security logs. The tool uses machine learning libraries such as TensorFlow and Scikit-learn for AI-driven insights, enabling predictive auditing and automated remediation. For efficient data storage and querying, Kloudraksha integrates with Redis and Vector DB, supporting fast, scalable, real-time monitoring. It also interfaces with reporting engines like JasperReports to generate detailed, actionable reports, and connects to external systems, such as SIEM tools, for broader threat management. Additionally, it supports API-based communication for email, SMS, and messaging notifications, ensuring users are promptly informed of critical audit findings. User authentication is handled through OAuth 2.0 or LDAP/Active Directory integrations, ensuring secure and role-based access to the tool's functionalities.

1. Cloud SDKs and APIs
2. Machine Learning Libraries
3. Database Interfaces
4. Reporting and Exporting Tools
5. Security and Compliance Tools
6. User Authentication Systems

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

Kloudraksha must be capable of processing real-time data from multiple cloud platforms with minimal latency. The system should perform continuous audits without negatively impacting cloud infrastructure performance, with a target response time of under 2 seconds for real-time monitoring and reporting queries.

3.4.2 Safety Requirements

Ensuring the safety of the system involves guaranteeing that the enhanced speech signal remains free from any unintended artifacts or distortions. This stipulation underscores the importance of maintaining the integrity of the audio output, preserving its clarity and naturalness throughout the noise cancellation process. Furthermore, it is imperative that the system does not inadvertently alter the meaning or context of the original speech signal. This requirement emphasizes the need for precision and fidelity in the noise cancellation algorithm, ensuring that the enhanced speech retains its intended message and conveys accurate information to the listener without any unintended modifications.

3.5 Scalability

The tool must scale horizontally to handle increasing cloud environments and configurations as organizations grow. Kloudraksha should support thousands of cloud assets across multiple platforms without performance degradation, leveraging microservices and cloud-native architecture.

3.5.1 Reliability and Availability

Kloudraksha must have 99.9% uptime, ensuring high availability for critical operations. The tool should be resilient to network failures and implement fault-tolerant mechanisms, such as data replication and automatic failover, to ensure continued operation during unexpected failures.

3.5.2 Security

The system must enforce strict security protocols, including encryption of data in transit and at rest using TLS and AES-256. Role-based access control (RBAC) should be implemented, along with multi-factor authentication (MFA) to secure user access. Regular security audits should be conducted to ensure compliance with security best practices.

3.5.3 Compliance

The tool must support compliance with industry standards like GDPR, HIPAA, and PCI-DSS. It should be easily adaptable to new or updated compliance regulations, ensuring it remains aligned with evolving industry requirements.

3.6 Analysis Models

3.6.1 SDLC Model

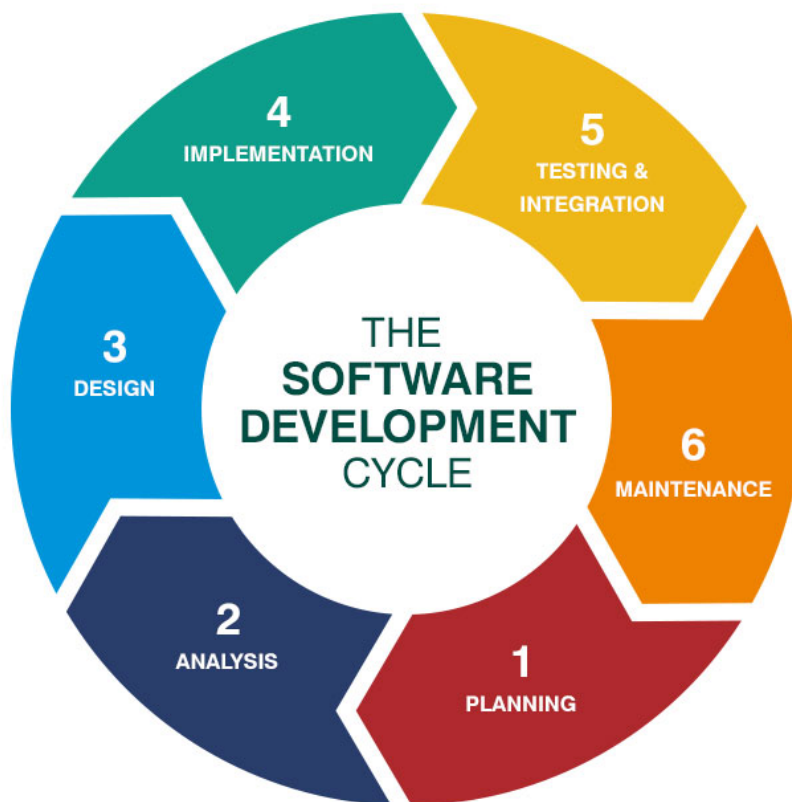


Figure 3.1: SDLC Model

CHAPTER 4

SYSTEM DESIGN

4.1 Model Architecture

The system design of Kloudraksha encompasses various architectural components and their interactions, ensuring a robust, scalable, and efficient cloud auditing solution. Below is an overview of the key components and their roles in the overall architecture. Kloudraksha follows a microservices architecture, allowing for modular development, scalability, and independent deployment of components. This design ensures that different functionalities (such as data collection, analysis, reporting, etc.) can be scaled and managed separately. The system operates primarily in the cloud, leveraging cloud-native technologies and services for performance and resilience.

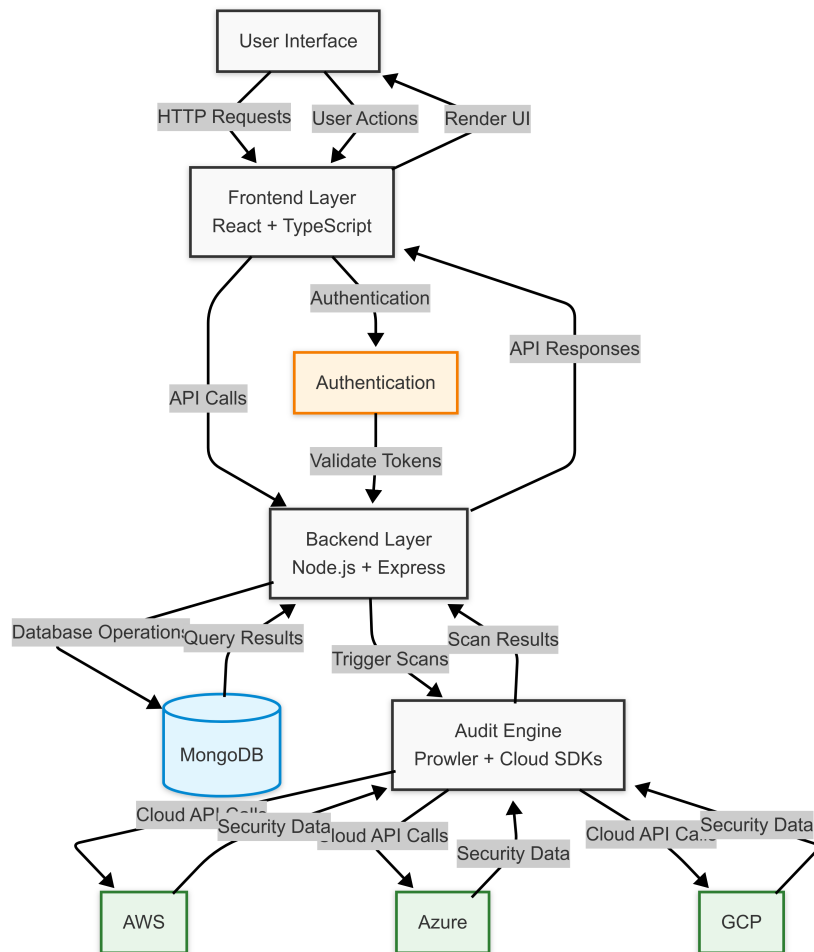


Figure 4.1: System Architecture

4.2 Key Components

4.2.1 Multi-cloud Integration Layer

This layer is responsible for connecting to various cloud platforms (AWS, Azure, Google Cloud) through their APIs.

Functionality: It collects configuration data, security logs, and compliance-related information from each cloud provider. The integration layer handles authentication, data retrieval, and error management.

4.2.2 Real-time Monitoring Engine

This component continuously scans cloud configurations for vulnerabilities and compliance issues.

Functionality: Utilizing AI/ML algorithms, it analyzes real-time data to detect misconfigurations and anomalies, triggering alerts for immediate action.

4.2.3 Compliance Engine

Responsible for validating cloud configurations against industry-specific regulatory standards (e.g., GDPR, HIPAA, PCI-DSS).

Functionality: It offers customizable compliance checks and generates reports that indicate compliance status and necessary remediation actions.

4.2.4 User Dashboard

A web-based interface that presents audit reports and insights.

Functionality: The dashboard offers customizable views, visualizations of audit findings, and easy navigation for both technical and non-technical users.

4.2.5 Authentication and User Management

Handles user login, role assignments, and access control.

Functionality: Implements OAuth 2.0/OpenID Connect for secure authentication, supporting multi-factor authentication (MFA) for enhanced security.

4.3 Data Flow

4.3.1 Data Collection

The integration layer fetches configuration data and security logs from connected cloud platforms using APIs at regular intervals or in real-time.

4.3.2 Monitoring and Analysis

The real-time monitoring engine analyzes the collected data for vulnerabilities and compliance violations, leveraging AI/ML algorithms.

4.3.3 Compliance Checks

The compliance engine validates the configurations against regulatory standards and generates compliance reports.

4.3.4 Remediation Actions

Upon detection of misconfigurations, the automated remediation module suggests corrective actions or applies fixes automatically.

4.3.5 User Notification

The notification system alerts users of significant findings, ensuring timely remediation.

4.3.6 Report Generation

The reporting engine creates detailed audit reports that users can access via the dashboard or export to share with stakeholders.

4.4 Deployment and Scalability

Kloudraksha will be deployed in a cloud environment (e.g., AWS, Azure, or Google Cloud), utilizing containerization (e.g., Docker) and orchestration tools (e.g., Kubernetes) for managing microservices. This setup allows for horizontal scaling of components as demand increases, ensuring that the system can efficiently handle large-scale cloud environments.

4.5 Security Considerations

The system design incorporates robust security measures, including:

1. Encryption of data in transit and at rest.
2. Role-based access control (RBAC) to manage user permissions.
3. Regular security audits and vulnerability assessments to identify and mitigate risks.

CHAPTER 5

PROJECT PLAN

5.1 UML Diagrams

5.1.1 Class Diagram

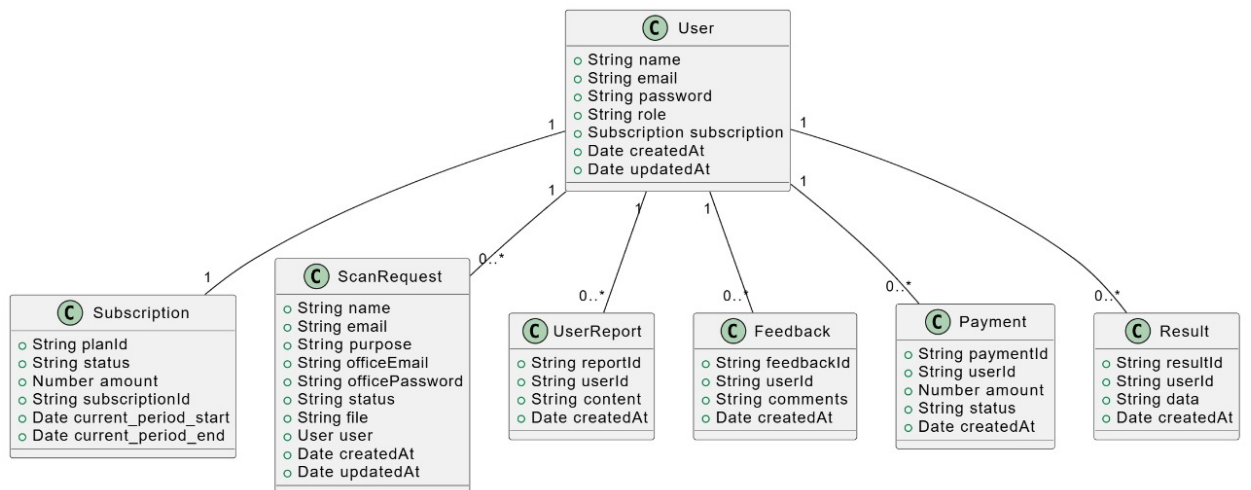


Figure 5.1: Class Diagram

5.1.2 Use Case Diagram

5.1.3 Sequence Diagram

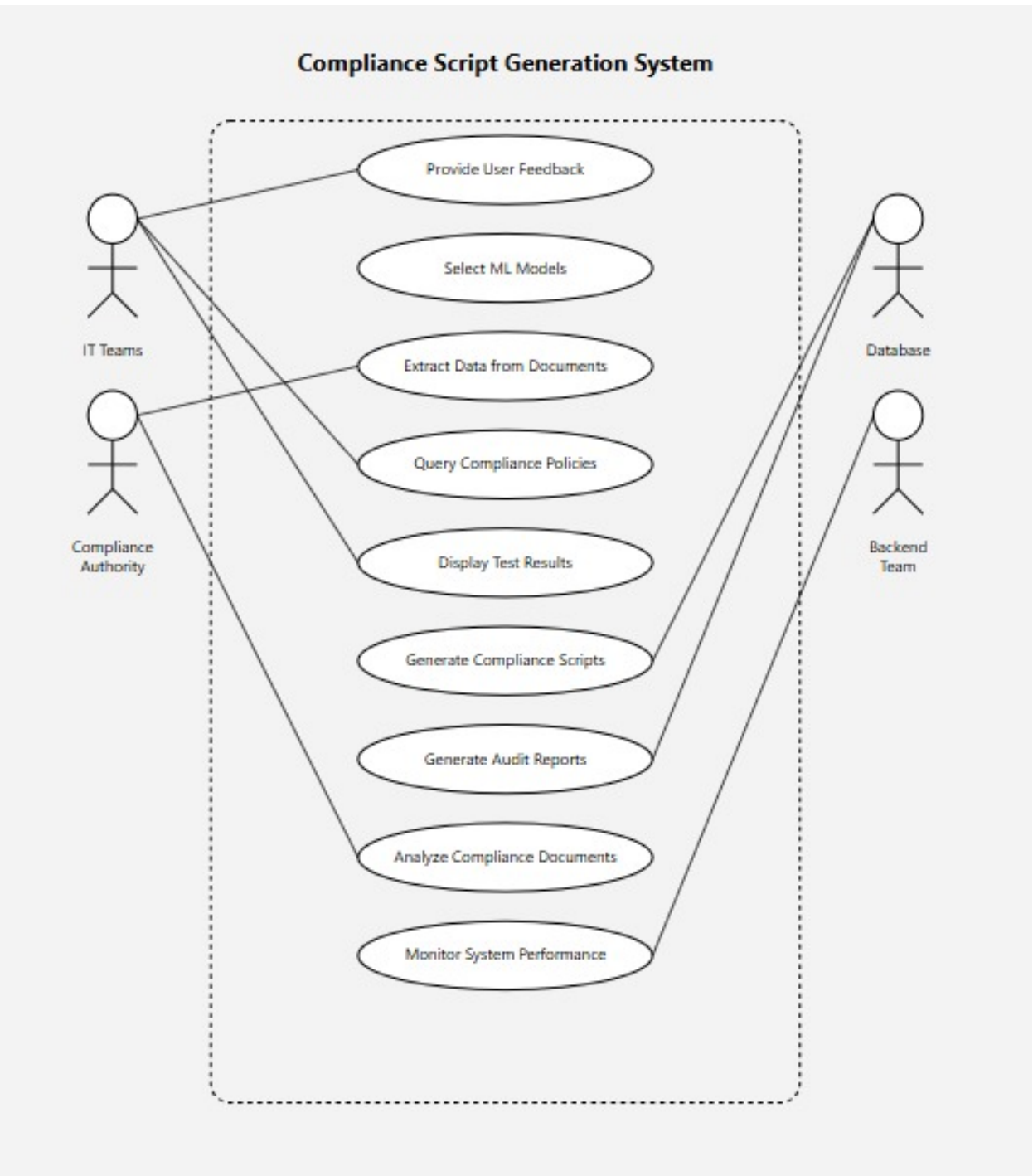


Figure 5.2: Use Case Diagram

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

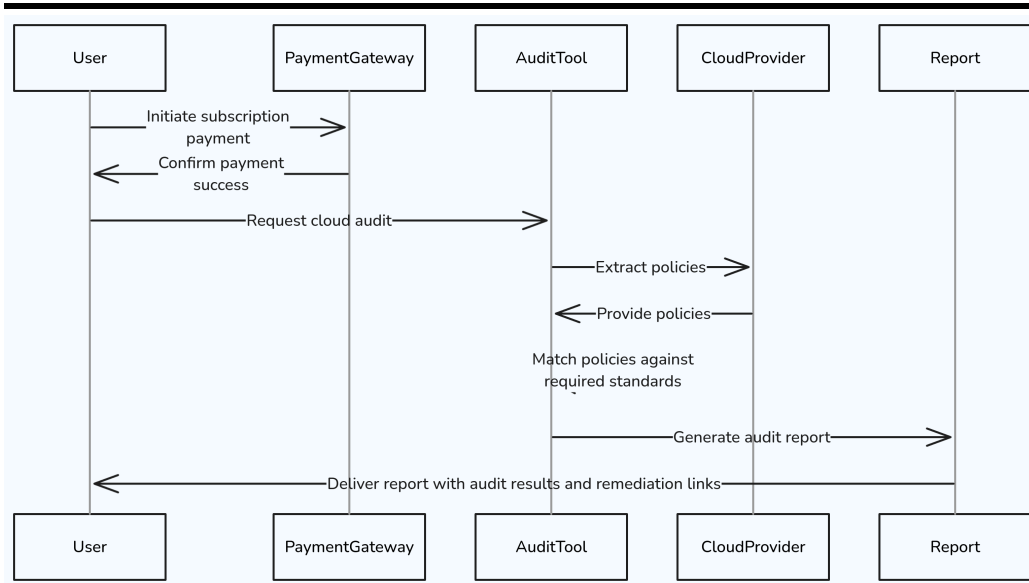


Figure 5.3: Sequence Diagram

5.2 Project Estimate

5.2.1 Reconciled Estimates

- **Research and Development:** Estimated a total of 6 months to develop and optimize the multi-cloud auditing framework and automation features. This includes extensive research into existing cloud auditing tools, identifying gaps, and developing innovative solutions tailored to Kloudraksha's objectives. It also accounts for the integration of AI-driven insights and automated remediation capabilities.
- **Testing and Evaluation:** Estimated 4 months to conduct thorough testing of Kloudraksha's functionalities, including performance evaluation, security assessments, and compliance checks. This phase involves setting up various testing scenarios, collecting and analyzing performance data, and fine-tuning the tool for optimal results.
- **Documentation and Review:** Estimated 3 months to provide comprehensive documentation, ensuring effective knowledge transfer among team members and stakeholders. This includes regular reviews, updates, and presentations to senior leadership to expedite the requirements specification process and initiate feedback loops. Additionally, efforts will be made to publish findings in relevant journals and conferences to disseminate knowledge.
- **Budget:** The budget includes salaries and benefits for the project team members, factoring in their roles and responsibilities. A consoli-

dated budget will cover all estimated expenses and include a buffer for unforeseen costs, ensuring financial stability throughout the project.

Based on these estimates, the total project timeline for developing Kloudraksha would be approximately 13 months. However, these estimates are indicative, and actual timelines may vary based on specific project requirements, resource constraints, and availability.

5.2.2 Project Resources

1. **Cloud Security Engineers (3):** Responsible for designing, developing, and optimizing the multi-cloud auditing capabilities of Kloudraksha. Their tasks include integrating with various cloud platforms (AWS, Azure, Google Cloud), implementing real-time monitoring features, and ensuring compliance with industry regulations. They will also work closely with the development team to enhance the tool's functionalities based on user feedback.
2. **Data Analysts (1):** Tasked with analyzing cloud configuration data to inform the development of Kloudraksha. They will clean, preprocess, and extract relevant features from the data, ensuring high-quality input for the auditing algorithms. Data analysts will also conduct exploratory data analysis to identify trends and anomalies, guiding the development of the tool and performance evaluation metrics.
3. **User Experience (UX) Designers (2):** Focused on creating an intuitive user interface for Kloudraksha. They will design wireframes and prototypes, conduct user testing, and iterate based on feedback to ensure the tool is accessible and usable for both technical and non-technical stakeholders.
4. **Project Management Tool:** Google Workspace
5. **Collaboration Tools:** GitHub, Google Colab, Google Meet, Discord

5.2.3 Project Timeline



Figure 5.4: Project Timeline

5.3 Risk Management

5.3.1 Risk Identification

- **Technical Risk:**
The complexity of multi-cloud integration could lead to delays or compatibility issues.
 1. Difficulties in automating real-time auditing across different cloud platforms due to varying APIs and configurations.
 2. Challenges in ensuring comprehensive compliance with industry regulations, which may require extensive customization.
 3. Potential performance issues related to the scalability of the solution in handling large volumes of data.
- **Financial Risk:** Budget overruns could occur due to unexpected expenses related to software licensing, cloud service costs, or additional resource requirements.
- **Operational Risk:** Unforeseen changes in project scope, such as the addition of new cloud platforms or compliance requirements, could disrupt the project timeline.
- **Resource Risk:** The risk of key personnel leaving the project team, leading to knowledge gaps and potential delays in development.

5.3.2 Risk Analysis

- **Technical Risk:** This risk is assessed as high impact due to its potential to delay the project significantly, with a medium likelihood of occurrence. To mitigate this risk, a focus on robust architecture design and modular development will be emphasized.
- **Financial Risk:** The financial risk is evaluated as medium impact, with a low likelihood of occurrence. To mitigate this risk, ongoing financial reviews and a strict budget approval process will be enforced.
- **Operational Risk:** The operational risk is considered medium impact with a medium likelihood. To mitigate this risk, regular stakeholder engagement and communication will be implemented to ensure alignment on project scope.
- **Resource Risk:** This risk is deemed high impact, as losing key personnel could lead to significant delays, with a medium likelihood of occurrence. Mitigation strategies include cross-training team members and maintaining detailed documentation of the project.

5.3.3 Overview of Risk Mitigation, Monitoring, Management

- **Technical Risk Mitigation:** A focus on modular design will facilitate easier integration across platforms, and extensive documentation will support the development process.
- **Financial Risk Mitigation:** A rigorous budget review process, including regular financial reporting, will help monitor expenditures closely and address any overruns early.
- **Operational Risk Mitigation:** A formal change management process will be in place to assess the impact of any proposed changes thoroughly before approval.
- **Resource Risk Mitigation:** Cross-training team members will ensure continuity in case of personnel changes, and knowledge-sharing sessions will be regularly conducted.
- **Risk Monitoring:** Bi-weekly risk assessment meetings will be scheduled to review the status of identified risks and discuss any emerging issues.
- **Risk Management:** A contingency fund of 10% of the total budget will be allocated to address unforeseen challenges that may arise during the project lifecycle.

5.3.4 Project Task Set

- Set up the necessary cloud environments, development tools, and APIs for multi-cloud auditing integration.
- Conduct an initial survey of cloud auditing tools and assess existing platforms (AWS, Azure, GCP) for integration potential.
- Research multi-cloud support, real-time monitoring capabilities, and compliance management frameworks to identify key features for Kloudraksha.
- Test the integration feasibility of selected cloud platforms and APIs, and evaluate the complexity of multi-cloud configuration.
- Finalize the architecture for Kloudraksha, including the design for real-time monitoring, compliance validation, and reporting workflows.
- Set up Redis and Vector DB for real-time data storage and retrieval to support large-scale cloud audits.
- Prepare APIs and integrate AWS, Azure, and Google Cloud with the Kloudraksha system for multi-cloud support.
- Begin development of the real-time monitoring engine and compliance validation module.
- Implement and test AI/ML algorithms for automated detection of cloud misconfigurations and vulnerabilities.
- Analyze performance of real-time auditing and compliance features across different cloud environments.
- Iteratively refine the AI/ML algorithms and optimize the system for scalability, speed, and accuracy.
- Document the entire project process, including design choices, methodologies, findings, and outcomes.
- Present the project results to stakeholders and plan for further development, including potential integration with additional cloud providers or services.

5.4 Team Organization

5.4.1 Team structure

- **Ameya Surana:** Backend Developer.

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

- **Mustafa Trunkwala:** Cloud Specialist, Backend Developer, System Designer.
- **Riya Wani:** Frontend Development, UI/UX Design.
- **Prof. Bhumesh Masram:** Internal mentor and guide.
- **Mr. Mandar Waghmare (Newtons Apple):** Project manager
- **Newtons Apple:** Project Sponsor and Industry connect.

5.4.2 Management reporting and communication

- **Management:** Jira is used for workflow management and efficiently tracking task completion and allocating tasks. Git, GitHub, and Google Colab were used to efficiently collaborate on the code.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

Kloudraksha comprises a modular and scalable architecture divided into multiple key components:

1) Frontend Module: Built with React and TypeScript, the frontend offers a user-friendly, responsive interface that facilitates real-time dashboards, audit management, policy customization, and report generation. It uses Redux for state management and Tailwind CSS for styling. The frontend allows users to trigger audits, handle configurations, and display technical and non-technical reports. Axios is used for asynchronous communication with the backend through RESTful APIs.

2) Backend Module: Node.js and Express.js were used to develop the backend, which is the core logic handler of the application. It has API endpoints for frontend interaction, handles audit scheduling and execution, and processes gathered data to detect security loopholes. MongoDB is the main data storage system with Mongoose being used for object modeling. JWT-based authentication and role-based access control (RBAC) are implemented for secure authorization and protection of resources.

3) Audit Script Engine: This engine runs audit scripts against cloud provider APIs like AWS SDK, Azure SDK, and GCP Client Libraries. The engine runs in a sandboxed JavaScript environment and checks cloud resource configurations for compliance with predefined rules and compliance standards. The engine is extensible, and new scripts and providers can be added. Scripts verify configurations such as public exposure of S3 buckets, EC2 security group rules, IAM permissions, and encryption enforcement. The engine is containerized with Docker for scalability and reproducibility.

4) Data Storage and Management: MongoDB stores audit data, policies, user information, and cloud account settings. The schema has collections for audit results, policies, cloud accounts, and users. The system is optimized with indexing, data partitioning, and aggregation pipelines for efficient and quick querying and reporting.

5) Integration Deployment: Each component is containerized with Docker and orchestrated through Docker Compose for multi-container orchestration. Kloudraksha can be deployed on cloud platforms (AWS, Azure, GCP), on-premise with Kubernetes or Docker Swarm, or local for testing.

6) Security Features: Security features involve JWT for authentication, RBAC for access control, HTTPS for secure communication, and encrypted data storage. All important actions are logged for auditing.

6.2 Tools and Technologies Used

1) Frontend:

React: Development of UI using component-based design.

TypeScript: Provides type safety to JavaScript, enhancing maintainability of code.

Redux: State management to have predictable data flow.

Tailwind CSS: Utility-first CSS framework for responsive UI.

Axios: Takes care of asynchronous API communication.

2) Backend:

Node.js: JavaScript runtime for server-side.

Express.js: Web framework for development of APIs.

MongoDB: NoSQL database to store unstructured audit data.

Mongoose: ODM for MongoDB.

JWT: JSON Web Tokens to provide secure authentication of users.

Middleware: Employed for logging, validation, error handling.

3) Audit Engine Scripting:

JavaScript: For coding and running audit scripts.

Cloud SDKs: AWS SDK, Azure SDK, GCP APIs for API calls.

Docker: Isolation, consistency, and portability through containerization.

Docker Compose: Orchestration of multiple services.

4) DevOps Scalability Tools:

Kafka/RabbitMQ: Asynchronous processing and message queuing.

Redis/Memcached: Caching infrequently changing data.

HTTPS: Secure communication.

Kubernetes: Scalable on-prem deployments (optional).

6.3 Algorithm Details

The essential functionality of Kloudraksha is based on an auditing algorithm that performs predefined checks on cloud infrastructures. Below is a comprehensive description:

1) Script Execution:

1.1) Scripts are either in a DSL or JavaScript programming and reside in the database with versioning.

1.2) The audit script engine retrieves scripts and employs the SDKs of cloud providers to access the APIs of AWS, Azure, and GCP.

1.3) Each script retrieves configuration of target resources (e.g., S3, EC2, IAM, databases).

2) Rule Evaluation:

2.1) Scripts check against a defined security policy and compliance standard like CIS, NIST, GDPR, and HIPAA.

Examples:

- (1) Check if S3 buckets are publicly exposed.
- (2) Check that EC2 instances do not have default security groups being used.
- (3) Make sure IAM users adhere to the principle of least privilege.
- (4) Verify data in transit and data at rest encryption configurations.

3) Result Aggregation:

3.1) Processed results are compared to find non-compliance errors and security misconfigurations.

3.2) Violations are logged with metadata, severity, affected resources, and remediation actions.

4) Storage and Reporting:

4.1) Audit findings are stored in MongoDB in structured collections.

4.2) The reporting engine presents this data in the form of dashboards, downloadable reports, and compliance reports.

4.3) Reports are exposed via the frontend and are usable by both technical and non-technical stakeholders.

5.) Automation and Remediation:

5.1) Kloudraksha enables scheduled and ongoing audits.

5.2) It also provides features for automated remediation (e.g., revoking public access, applying encryption).

5.3) Real-time alerts and notifications can be set up to notify of new or severe vulnerabilities.

6) Scalability Optimization:

6.1) The audit engine executes multiple scripts concurrently with Docker containers.

6.2) Asynchronous queues isolate long-running tasks, avoiding UI delays and backend overload.

6.3) Microservices architecture and load balancing enable horizontal scaling during heavy loads.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

Kloudraksha has been thoroughly tested to ensure its functionality, performance, and scalability on multiple cloud infrastructures. The main types of tests performed are:

1) Functional Testing:

This verified that every aspect of Kloudraksha—e.g., starting audit, policy definition process, reporting generation, and dashboard refresh—functioned as expected. API endpoints, frontend functionality, and the audit script engine were tested for proper behavior in valid and invalid scenarios.

2) Integration Testing:

All modules that were wired together frontend, backend, and audit engine—were tested end-to-end to ensure end-to-end data communication, flow, and consistency. RESTful API exchange between frontend and backend and data transfer with MongoDB were cross-checked.

3) Performance Testing:

Kloudraksha was subjected to various load conditions to analyze system responsiveness and performance. Parameters such as audit execution time, CPU usage, and memory consumption were captured while auditing small to large cloud ecosystems.

4) Scalability Testing:

The scalability of the system to manage growing numbers of cloud resources (from hundreds to thousands) was evaluated. Kloudraksha was demonstrated to audit 10,000 cloud resources effectively, with linear scaling of running time.

5) Security Testing:

Security tests were conducted to provide assurance of the strength of authentication, authorization (JWT, RBAC), and safe storage/transmission of audit information. Sensitive user activities were audited to ensure audit traceability.

6) Usability Testing:

User opinions were gathered to determine UI simplicity, navigation ease, and report clarity. Technical and non-technical stakeholders tested the system for accessibility and understanding.

7) Accuracy Testing:

The audit engine was validated using a 50-security misconfiguration known dataset to validate vulnerability detection accuracy. This kind of testing ensured that actual issues were correctly identified while avoiding excessive false positives.

7.2 Test cases & Test Results

1) Audit Execution Speed Test:

Test Case:

Run an audit on a simulated AWS environment with 500 resources.

Expected Result:

Audit should complete in under 20 minutes.

Actual Result:

Audit completed in 15 minutes, significantly faster than manual audits (8 hours).

2) Vulnerability Detection Accuracy:

Test Case:

Introduce 50 known misconfigurations in AWS resources.

Expected Result:

At least 90% of issues should be detected.

Actual Result:

48/50 issues detected (96% accuracy), with 0% false positives, demonstrating high precision.

3) Scalability Audit Test:

Test Case:

Perform audits across cloud environments increasing from 100 to 10,000 resources.

Expected Result:

Linear increase in audit execution time.

Actual Result:

Time scaled linearly, confirming horizontal scalability without performance degradation.

4) User Access Control Test:

Test Case:

Attempt unauthorized access to audit results with a low-privilege user.

Expected Result:

Access should be denied.

Actual Result:

Access blocked successfully using RBAC and JWT validation.

5) Report Generation and Visualization:

Test Case:

Generate and view audit summary post-execution.

Expected Result:

Reports should be detailed, readable, and downloadable.

Actual Result:

Reports generated with compliance breakdown, remediation steps, and resource-level insights.

6) Frontend-Backend Communication:

Test Case:

Perform audit action via UI and fetch real-time results.

Expected Result:

Smooth and accurate UI update through API.

Actual Result:

Real-time dashboard update achieved via REST APIs with no lag or inconsistency.

7) Resource Utilization Monitoring:

Test Case:

Observe system performance during concurrent audits.

Expected Result:

CPU $\leq$ 60%, Memory $\leq$ 70%.

Actual Result:

CPU usage $\leq$ 50%, Memory usage $\leq$ 60%, indicating resource-efficient operation.

8) Multi-cloud Environment Testing:

Test Case:

Audit AWS, Azure, and GCP environments concurrently.

Expected Result:

All audits execute independently and successfully.

Actual Result:

All multi-cloud audits succeeded, maintaining isolation and accuracy.

9) Error Handling Validation:

Test Case:

Introduce invalid API keys and malformed audit policies.

Expected Result:

Error logs and meaningful messages should be returned.

Actual Result:

Appropriate errors logged and displayed, preventing crashes or silent failures.

10) User Feedback on Usability:

Test Case:

Conduct surveys post-usage among beta testers.

Expected Result:

Majority positive response on ease of use.

Actual Result:

High user satisfaction, especially for clarity in audit reports and custom policy options.

CHAPTER 8

RESULTS

8.1 Outcomes

The deployment and assessment of Kloudraksha produced a number of far-reaching effects along functional, technical, and user experience aspects. These effects confirm the efficacy of the tool in practical cloud security audit situations:

1) Substantial Audit Time Reduction:

Kloudraksha significantly shortened the duration to conduct cloud security audits. In comparative testing, Kloudraksha audited an AWS environment with 500 cloud resources in 15 minutes, while manual audits of the same environment took around 8 hours. This represents more than 95% reduction in audit time, allowing for quicker detection and mitigation of vulnerabilities and compliance issues.

2) High Accuracy in Vulnerability Detection:

The system was 96% accurate in detecting known security misconfigurations with 0% false positive rate in accuracy tests. In a test case of 50 intentionally misconfigured resources in AWS, 48 vulnerabilities were accurately detected. This degree of accuracy shows Kloudraksha's ability to detect true security issues without producing noise from irrelevant results.

3) Demonstrated Scalability:

Kloudraksha successfully audited environments scaling up to 10,000 cloud resources without performance degradation. Audit execution time increased linearly with the number of resources, proving the architecture's horizontal scalability. This ensures that the tool remains efficient and responsive, even for large enterprise-scale cloud deployments.

4) Enhanced Resource Utilization:

Performance testing showed that Kloudraksha is efficient in resources. Average CPU usage was less than 50% and memory utilization less than 60% even in peak-volume audits. This ensures that the application can be run effectively on low-key infrastructure without demanding high-end computing resources.

5) Better User Experience and Accessibility:

Feedback from beta users, which included technical and non-technical stakeholders, called attention to Kloudraksha's ease of use UI, streamlined navigation, and ease of reporting. The presence of both technical and non-technical reports, as well as visual dashboards and remediation recommendations, improved accessibility and allowed for the audit results to be actionable by a broad set of users.

6) Real-Time Monitoring and Continuous Compliance:

Kloudraksha's real-time dashboard functionality guaranteed up-to-the-minute compliance statuses and critical performance metrics (KPIs). This allowed for uninterrupted monitoring instead of snapshot views, giving organizations a constant perspective of their security posture and the ability to engage in proactive remediation.

7) Multi-Cloud Compatibility:

The tool effectively executed concurrent audits across AWS, Microsoft Azure, and Google Cloud Platform (GCP), validating its strong multi-cloud support. This feature streamlines security management for organizations that employ a heterogeneous cloud environment, allowing them to minimize the use of several disparate tools.

8) Flexible Policy Customization and Extensibility:

Organizations valued having the capacity to create and control personalized audit policies, which allowed for alignment with unique business requirements and industry regulations (CIS, NIST, HIPAA, GDPR). The extensible and modular audit engine allows for the addition of new cloud providers and rules, providing future-proof flexibility.

9) Security and Audit Integrity:

Kloudraksha's implementation of JWT-based authentication, role-based access control (RBAC), HTTPS communication, and audit logging provided a secure and dependable audit environment. Each crucial operation was logged, and sensitive operations were limited according to user role to maintain integrity and accountability.

10) Support for Future Expansion:

Effective deployment and validation of Kloudraksha paved the way for even more improvements, including support for other platforms (e.g., Kubernetes, Alibaba Cloud), AI-based predictive threat detection, and more extensive CI/CD pipeline integration. These results as a whole reinforce Kloudraksha as a complete, future-proofing cloud security auditing and compliance management solution.

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

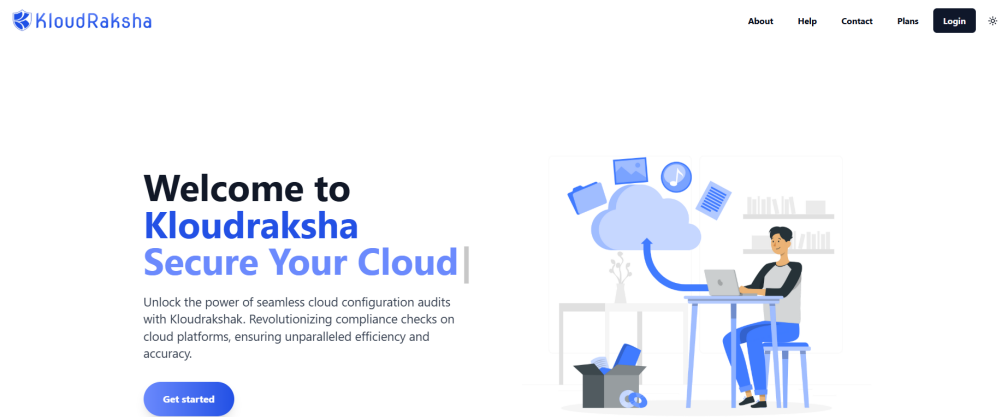


Figure 8.1: Hero section of website

The login screen features a white background with a light gray border. At the top, there are two tabs: "Sign Up" and "Login", with "Login" being the active tab. Below the tabs, the heading "Login" is followed by the instruction "Head toward your dashboard by entering your login credentials". The form includes an "Email" field with the placeholder text "mustafa@kloudraksha.com" and a "Password" field with a masked password "*****". Below the password field, there is a green checkmark icon and the text "I'm not a robot", followed by a reCAPTCHA widget. At the bottom of the form, there is a blue "Login" button.

Figure 8.2: Login Screen

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

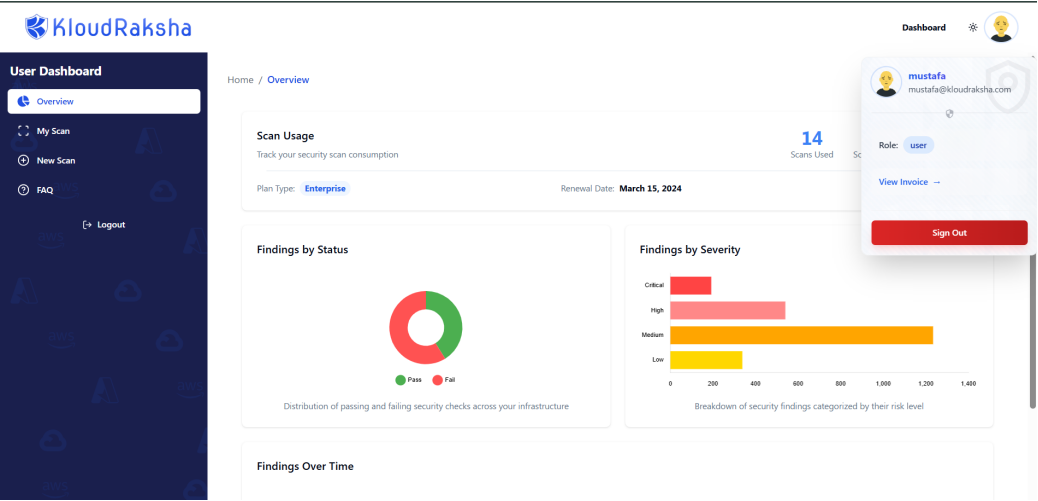


Figure 8.3: User Dashboard

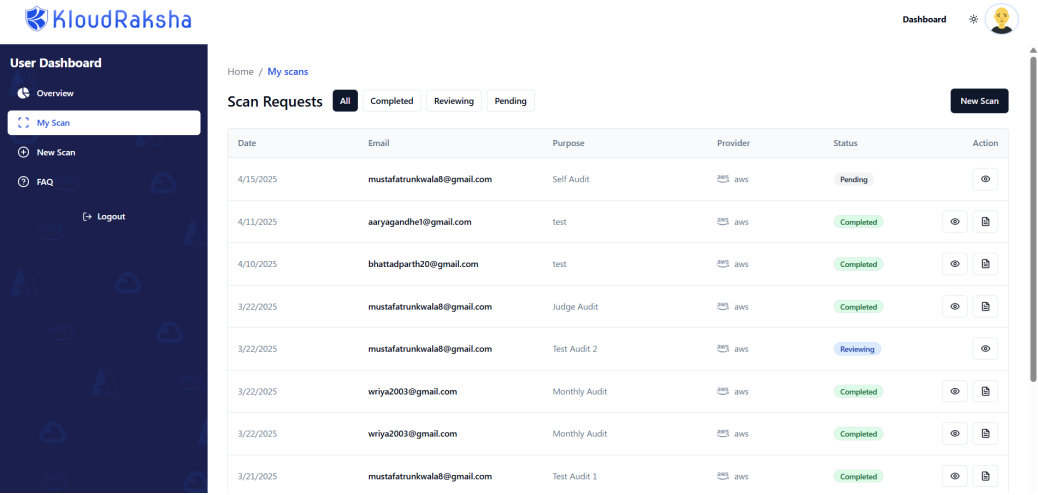


Figure 8.4: User Dashboard - My Scan

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

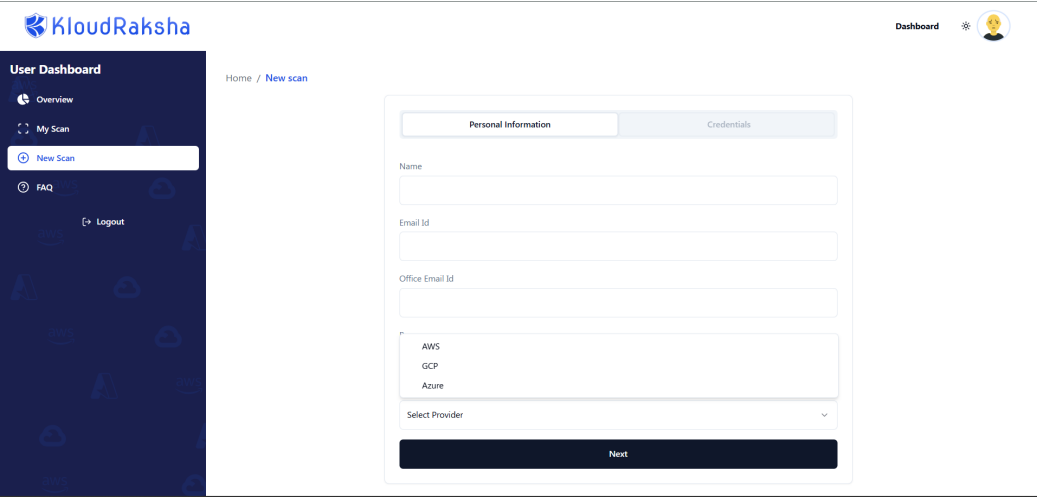


Figure 8.5: User Dashboard - New Scan

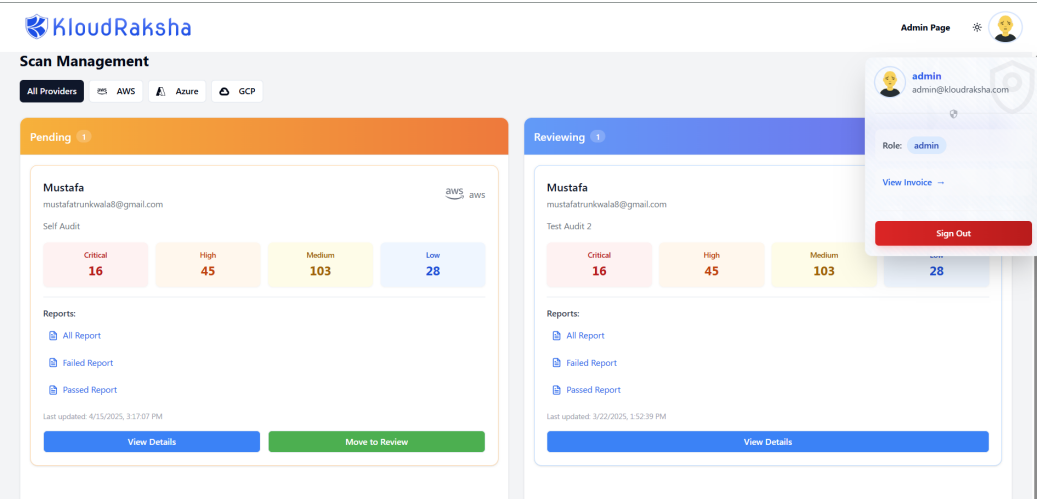


Figure 8.6: Admin Dashboard - Kanban Layout

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

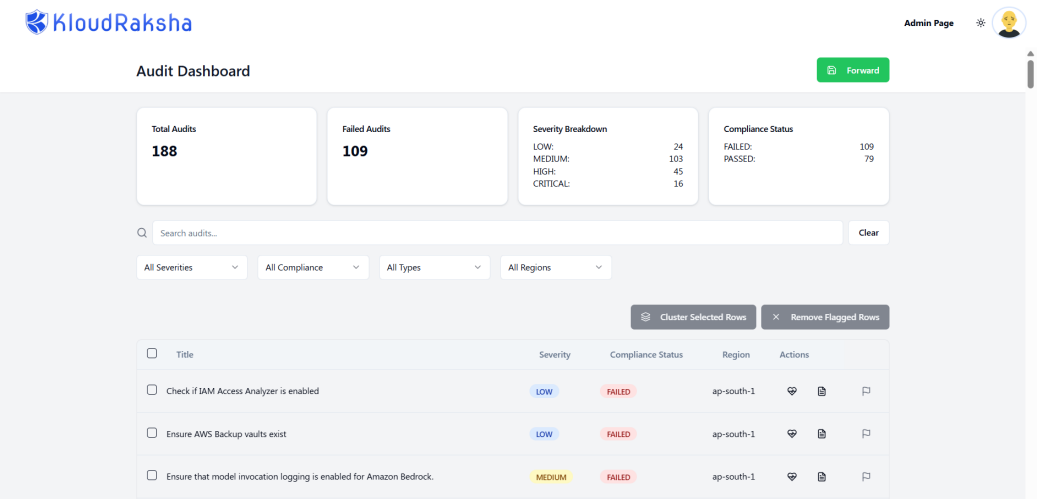


Figure 8.7: Audit Dashboard

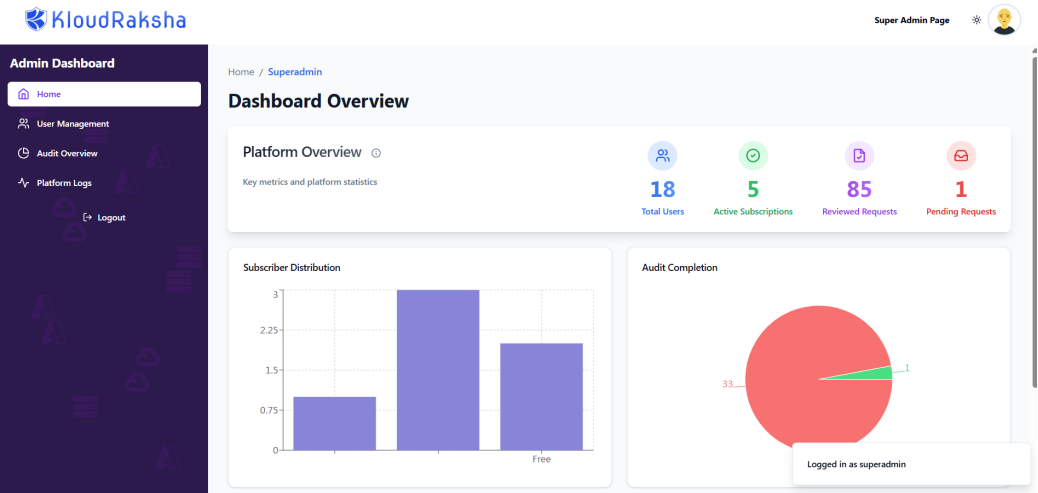


Figure 8.8: Super Admin Dashboard

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

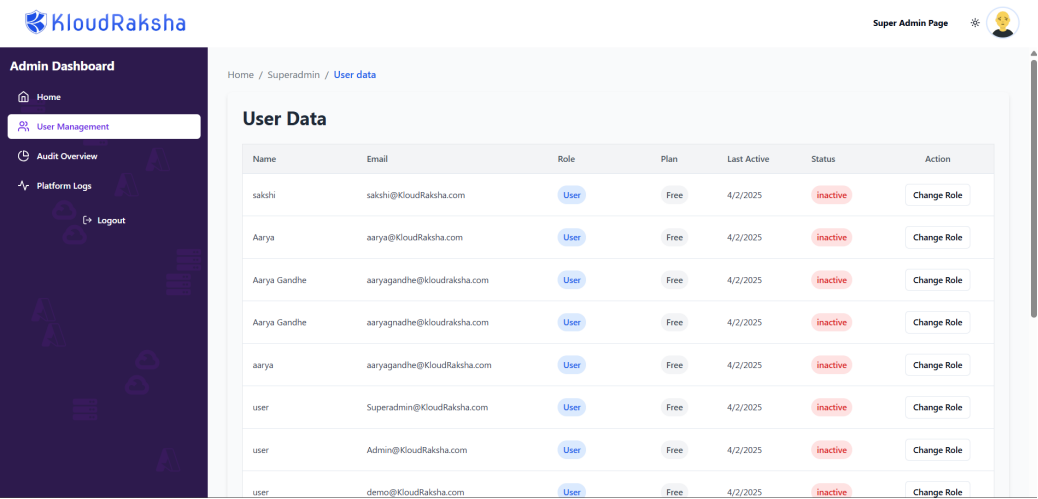


Figure 8.9: Super Admin Dashboard - User Management

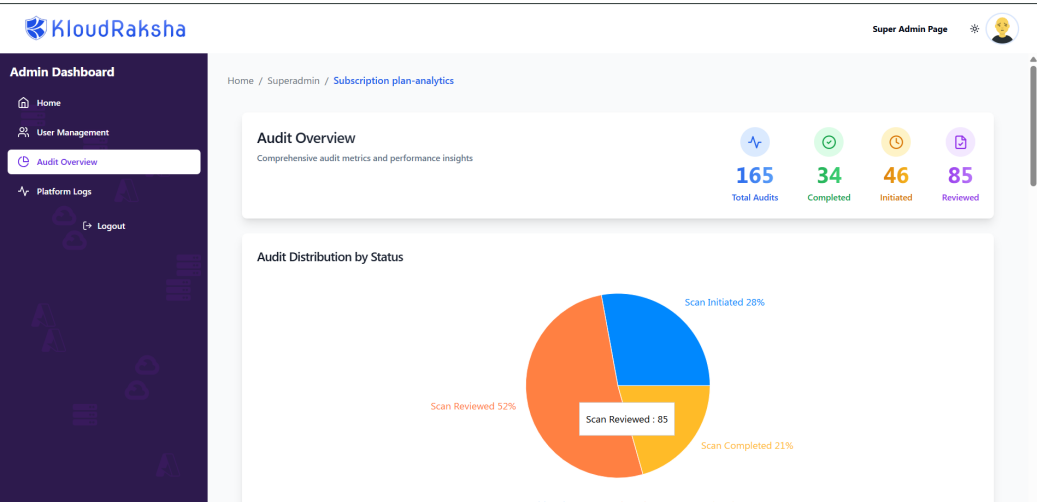


Figure 8.10: Super Admin Dashboard - Audit Overview

Kloudraksha: A Cloud Auditing Tool for Enhanced Security and Compliance

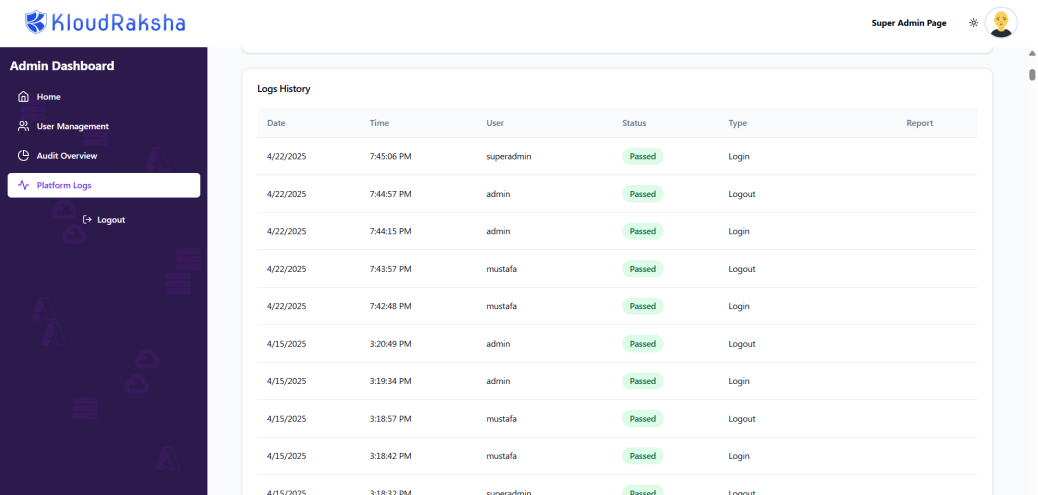


Figure 8.11: Super Admin Dashboard - Platform Logs

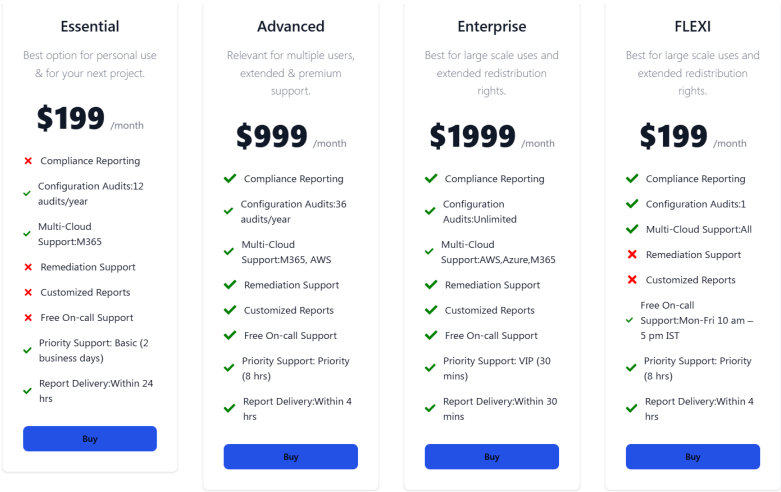


Figure 8.12: Subscription Model

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

Kloudraksha offers a transformative approach to cloud auditing, addressing key limitations in existing tools such as the lack of multi-cloud support, insufficient real-time auditing, and complex reporting. By integrating advanced AI/ML algorithms, it ensures continuous, automated audits across multiple cloud platforms like AWS, Azure, and Google Cloud. The solution simplifies compliance management and provides actionable insights that are accessible to both technical and non-technical stakeholders. Kloudraksha's scalability, real-time performance, and user-friendly reporting make it a critical tool for organizations looking to enhance their cloud security and compliance with minimal manual effort. Through its innovative architecture and design, Kloudraksha not only meets current industry needs but also sets the stage for future enhancements and broader cloud platform integration.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Survey Paper Details

Paper Status: Submitted and Under Review (Cleared review stage 2)
Name of Journal: PICT's International Journal of Engineering and Technology
Journal Webpage: <https://pijet.org/>

10.1.2 Implementation Paper Details

Paper Status: Submitted and Under Review
Name of Conference: 2025 Seventeenth International Conference on Contemporary Computing (IC3-2025)
Track Name: Data and Cloud Computing Track
Date of Conference: August 7-9, 2025
Conference Webpage: <https://www.ic3conf.net/index.html>

10.2 Appendix B

10.2.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.
Similarity: 3%

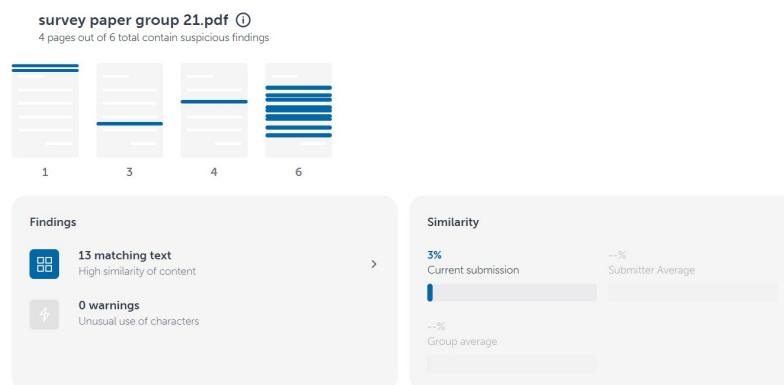


Figure 10.1: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Torkura, K. A., Meinel, C., Dossou, P. E. (2022). "Continuous Auditing Threat Detection in Multi-Cloud Infrastructure." Elsevier. This paper introduces the CSBAuditor system, focusing on continuous auditing in multi-cloud environments.
- [2] Ligatti, J., Bauer, L., Walker, D. (2016). "Cloud Security Auditing: Major Approaches and Existing Challenges." SpringerLink. This comprehensive review discusses traditional and emerging cloud auditing methodologies.
- [3] Wang, X., Ren, K., Lou, W., Li, L. (2023). "Security and Privacy of Cloud Data Auditing Protocols: A Review, State-of-the-art, Open Issues, and Future Research Directions." IJACSA. This paper focuses on cryptographic methods for secure cloud auditing, particularly homomorphic verifiable tag
- [4] "MEDINA Framework for Continuous Cloud Auditing." (2024). Open Research Europe. This study introduces the MEDINA framework, which supports continuous auditing through automated monitoring and Blockchain technology
- [5] Torkura, K. A., Luthi, M., Meinel, C., Dossou, P. E. (2022). "A Cloud Security Auditing Framework for Detecting Misconfigurations." Elsevier. This paper emphasizes the importance of secure baseline configurations for continuous auditing.
- [6] AMOE Tool Development Team. (2020). "AMOE: Automating Evidence Extraction for Continuous Cloud Auditing." ArXiv. The paper explores NLP techniques for extracting organizational evidence for audits.
- [7] Zissis, D., Lekkas, D. (2012). "Addressing cloud computing security issues." Future Generation Computer Systems, 28(3), 583-592. This paper provides an overview of security challenges in cloud computing, emphasizing the need for continuous monitoring systems.
- [8] Majumdar, S., et al. (2016). "Proactive Verification of Security Compliance for Clouds." Lecture Notes in Computer Science. This research focuses on proactive approaches to security compliance in virtualized cloud environments.
- [9] Lu, Z., Tang, Z., Li, R. (2013). "Efficient Public Batch Auditing Protocol for Data Security in Multi-Cloud Storage." ChinaGrid Annual Conference. This paper introduces a public auditing protocol for multi-cloud environments.
- [10] Narain, S. (2005). "Network Configuration Management via Model Finding." LISA Conference Proceedings. This work addresses the management and auditing of network configurations in cloud-based environments.

- [11] Leglaive, S., Alameda-Pineda, X., Girin, L., & Horaud, R. (2020, May). A recurrent variational autoencoder for speech enhancement. In ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 371-375). IEEE.